Calibration Algorithm Testbed: Simulation and Experimental Validation

A Minimal Model for Testing Gap-Filling Methods

March 2026

Abstract. We propose a two-track testbed — simulation and experiment — for validating the four gap-filling methods developed in our companion document. Track A is a Python-based *digital twin* of a transmon processor, built on QuTiP and scqubits, that models multi-level transmons with frequency-dependent T_1 (TLS defects), flux noise, readout noise, and tunable couplers. Track B defines a minimal experimental protocol on a 5–10 qubit testbed. For each of the four gaps (Optimus DAG, Snake optimizer, TLS forecasting, control electronics), we specify the simulation model, experimental protocol, benchmark metrics, and iteration procedure. The simulation enables rapid prototyping ($\sim 10^4$ calibration cycles/hour) before committing to cryostat time.

Contents

I. Overview . . . 3
- I.A. Two-Track Approach . . . 3
- I.B. What We Are Testing . . . 3

II. Track A: Simulation Model . . . 3
- II.A. Transmon Hamiltonian . . . 3
- II.B. Noise Model . . . 4
- II.C. Simulation Architecture . . . 6

III. Testing Gap 1: DAG Traversal . . . 7
- III.A. Simulation Test Protocol . . . 7
- III.B. Benchmark Metrics for Gap 1 . . . 8

IV. Testing Gap 3: Snake Optimizer . . . 9
- IV.A. Simulation Test Protocol . . . 9
- IV.B. Benchmark Metrics for Gap 3 . . . 10

V. Testing Gap 5: TLS Forecasting . . . 11
- V.A. Simulation Test Protocol . . . 11
- V.B. Benchmark Metrics for Gap 5 . . . 12

VI. Testing Gap 10: Electronics Noise Budget . . . 13
- VI.A. Simulation Test Protocol . . . 13

VII. Integration Test: Full Calibration Stack . . . 13
- VII.A. End-to-End Test . . . 13

VIII. Iteration Workflow 15
VIII.A. Development Cycle 15
VIII.B. Key Design Principles 15
IX. Minimal Hardware Requirements 16
IX.A. Simulation Track 16
IX.B. Experimental Track 16
X. Timeline 17
XI. Appendix: Software Architecture 17

I. Overview

I.A. Two-Track Approach

	Track A: Simulation	**Track B: Experiment**
Purpose	Rapid prototyping, algorithm debugging, parameter sweeps	Ground-truth validation, hardware-specific tuning
Platform	Python (QuTiP + scqubits + NumPy)	Dilution refrigerator + control electronics
Scale	1–20 qubits (Hamiltonian); 100+ (noise model only)	5–10 qubits (initial); 20+ (scaled)
Speed	$\sim 10^4$ calibration cycles/hour	$\sim$ 10–100 cycles/hour
TLS	Synthetic (configurable)	Real (device-specific)
Cost	Desktop CPU/GPU	Cryostat time

Table 1: Comparison of the two testbed tracks.

I.B. What We Are Testing

Each gap-filling method maps to specific testable components:

Gap	**Component Under Test**	**Simulation Test**	**Experimental Test**
1	DAG traversal: node ordering, tolerances, timeouts	Synthetic drift + DAG scheduler	Full bring-up on testbed chip
3	Snake optimizer: inner loop, error model, weight training	Synthetic frequency landscape + optimizer	CZXEB on multi-qubit device
5	TLS forecasting: 6 algorithms	Synthetic TLS with known trajectories	T_1 monitoring on real device
10	Electronics noise budget	Noise injection into simulation	Measure gate fidelity vs. noise floor

Table 2: Mapping of gap-filling methods to testbed components.

II. Track A: Simulation Model

II.A. Transmon Hamiltonian

The core simulator models N transmon qubits with L levels each (typically $L = 3$ to capture leakage). The single-qubit Hamiltonian for qubit i:

$$H_i = \sum_{k=0}^{L-1} \left(k\omega_i + \binom{k}{2} \alpha_i \right) |k\rangle\langle k|$$

where ω_i is the $|0\rangle \to |1\rangle$ transition frequency and α_i is the anharmonicity.

For flux-tunable transmons, the frequency depends on external flux:

$$\omega_{i(\Phi)} = \omega_i^{\max}\sqrt{|\cos(\pi\Phi/\Phi_0)|}$$

The coupling Hamiltonian between qubits i and j:

$$H_{ij} = g_{ij}\left(a_i^\dagger a_j + a_i a_j^\dagger\right)$$

with tunable coupling $g_{ij}(\Phi_c)$ for tunable-coupler architectures.

II.A.1. Implementation

```
# Core dependencies
import qutip as qt          # Lindblad master equation solver
import scqubits as scq      # Transmon spectrum computation
import numpy as np

class TransmonSimulator:
    """Minimal transmon processor simulator."""
    def __init__(self, n_qubits, n_levels=3):
        self.n_qubits = n_qubits
        self.n_levels = n_levels
        # Per-qubit parameters (randomized around design values)
        self.f01 = 5.0 + 0.5 * np.random.randn(n_qubits)      # GHz
        self.alpha = -0.20 + 0.01 * np.random.randn(n_qubits)  # GHz
        self.T1 = 50 + 20 * np.random.randn(n_qubits)          # us
        self.T2 = 60 + 20 * np.random.randn(n_qubits)          # us
```

II.B. Noise Model

The noise model is the critical component—it must be realistic enough that calibration algorithms trained in simulation transfer to hardware.

II.B.1. Frequency-Dependent T_1 (TLS Landscape)

SIMULATION. We model $T_1(f,t)$ as a background rate plus Lorentzian dips from N_{TLS} synthetic defects:

$$\Gamma_1(f,t) = \Gamma_{1,\text{bg}}(f) + \sum_{k=1}^{N_{\text{TLS}}} \frac{g_k^2\gamma_k}{\gamma_k^2 + \left(f - f_{\text{TLS},k}(t)\right)^2} \cdot s_{k(t)}$$

where each TLS has: frequency $f_{\text{TLS},k}(t)$ undergoing diffusion with $D = 2.2$ MHz/h$\frac{1}{2}$, coupling $g_k \sim \text{LogNormal}(1, 0.5)$ MHz, linewidth $\gamma_k \sim \text{Uniform}(0.05, 0.5)$ MHz, and switching state $s_{k(t)} \in \{0, 1\}$ with rate $r_k \sim \text{LogUniform}(10^{-4}, 1)$ Hz.

EXPERIMENT. Measure T_1 at 50–100 frequency points across the qubit's tunable range. Repeat every 2–4 hours to build a time-resolved TLS map. This is swap spectroscopy.

II.B.2. Flux Noise (Dephasing)

SIMULATION. Model $\frac{1}{f}$ flux noise with amplitude $A_\Phi = 3 \times 10^{-6}\ \Phi_0/\sqrt{\mathrm{Hz}}$. The dephasing rate at frequency f:

$$\Gamma_{\varphi(f)} = |d\omega/d\Phi|_f \cdot \sqrt{A_\Phi \cdot \ln\left(t_{\mathrm{gate}}/t_{\mathrm{IR}}\right)}$$

This is maximum away from the sweet spot and zero at $f = f_{\max}$.

EXPERIMENT. Measure T_2^* via Ramsey at multiple flux bias points. Extract A_Φ from the parabolic dependence $\frac{1}{T_2^*}(\Phi) \propto |d\frac{\omega}{d}\Phi|$.

II.B.3. Readout Noise

SIMULATION. Model as a confusion matrix per qubit:

$$M = \begin{pmatrix} 1-\varepsilon_0 & \varepsilon_1 \\ \varepsilon_0 & 1-\varepsilon_1 \end{pmatrix}$$

with $\varepsilon_0 \sim \mathrm{Uniform}(0.5\%, 2\%)$, $\varepsilon_1 \sim \mathrm{Uniform}(1\%, 5\%)$, correlated with dispersive shift χ and readout power.

EXPERIMENT. Measure confusion matrix via repeated single-shot classification with qubit prepared in $|0\rangle$ and $|1\rangle$.

II.B.4. Gate Errors

SIMULATION. Single-qubit gate error = coherence-limited error + calibration error:

$$\varepsilon_{1\mathrm{Q}} = \frac{t_g}{2T_1} + \frac{t_g}{2T_\varphi} + \delta\frac{\theta^2}{2} + \delta\beta^2 \cdot \left(\frac{\omega_R}{\alpha}\right)^2$$

where $\delta\theta$ is the rotation-angle miscalibration and $\delta\beta$ is the DRAG error. Two-qubit CZ error includes conditional-phase error, leakage, and stray-coupling contributions.

II.B.5. Parameter Drift

SIMULATION. Inject realistic drift into all parameters:

- Qubit frequency: Ornstein-Uhlenbeck process with $\sigma = 25$ kHz, $\tau = 6$ h.
- π-pulse amplitude: random walk with Allan deviation feature at ~ 1000 s.
- T_1: TLS-driven telegraph noise (see above).
- Readout: slow drift $\sigma = 0.1\%$/hour.

EXPERIMENT. All of these drifts are naturally present. Log all calibration parameters with timestamps to build a drift database.

II.C. Simulation Architecture

```
class ProcessorModel:
    """Full processor model with noise, drift, and TLS."""
    def __init__(self, n_qubits, topology="linear"):
        self.qubits = [QubitModel(i) for i in range(n_qubits)]
        self.couplers = build_couplers(n_qubits, topology)
        self.tls_landscape = TLSLandscape(n_qubits, n_tls_per_qubit=5)
        self.clock = SimClock()  # tracks simulated wall-clock time

    def step_time(self, dt_hours):
        """Advance all drift/TLS processes by dt_hours."""
        self.tls_landscape.diffuse(dt_hours)
        for q in self.qubits:
            q.drift_frequency(dt_hours)
            q.update_T1(self.tls_landscape)

    def measure_T1(self, qubit_idx, frequency=None):
        """Simulate a T1 measurement with realistic noise."""
        ...

    def run_gate(self, gate_type, qubits, params):
        """Simulate a gate and return the fidelity."""
        ...

    def run_xeb(self, qubit_pairs, n_cycles, n_circuits):
        """Simulate cross-entropy benchmarking."""
        ...
```

The key design principle: the simulation exposes the *same API* as the real hardware interface. Calibration algorithms are written once and run against either backend.

III. Testing Gap 1: DAG Traversal

III.A. Simulation Test Protocol

SIMULATION.

Setup: Initialize a 10-qubit `ProcessorModel` with randomized parameters. Implement the 52-node DAG from the gap-solutions document.

Test 1 — Cold Start: Run the full DAG from scratch (all nodes uncalibrated). Measure: total wall-clock time, number of failed nodes, final gate fidelities.

Test 2 — Maintenance Under Drift: After initial calibration, advance simulated time in 15-minute steps. At each step, call `maintain(target_node)` on the top-level node. Measure: number of recalibrations triggered, fidelity stability over 24 simulated hours.

Test 3 — Diagnosis: Inject a sudden parameter failure (e.g., T_1 drops by 50% on one qubit). Measure: time to detect, correct diagnosis path, recovery fidelity.

Test 4 — Tolerance Sweep: Sweep tolerance values for each node and measure the trade-off between recalibration frequency and fidelity stability. Identify the Pareto frontier.

Test 5 — Timeout Sweep: Vary timeout periods and measure the fraction of `check_state` calls that correctly avoid unnecessary recalibration (true negative rate) vs.
missed drift events (false negative rate).

EXPERIMENT.

Protocol: Deploy the DAG on a 5–10 qubit chip. Run the full bring-up sequence. Measure total time and success rate. Then run maintenance for 24+ hours continuously, logging every `check_state`, `check_data`, and `calibrate` call with timestamps and outcomes.

Metrics: Compare against manual calibration baseline (human expert or existing framework like Qibocal).

III.B. Benchmark Metrics for Gap 1

Metric	**Definition**	**Target**
Bring-up time	Wall-clock from cold start to all nodes passing	< 30 min (10Q)
Bring-up success rate	Fraction of qubits fully calibrated	$> 95\%$
Maintenance overhead	Fraction of time spent recalibrating vs. computing	$< 10\%$
Drift detection latency	Time from parameter shift to recalibration trigger	$< 2 \times$ timeout
False positive rate	Unnecessary recalibrations / total checks	$< 5\%$
Diagnosis accuracy	Correct root cause identified / total diagnoses	$> 90\%$
Fidelity stability (σ)	Std. dev. of gate fidelity over 24 h	$< 0.05\%$

Table 3: Benchmark metrics for DAG traversal testing.

IV. Testing Gap 3: Snake Optimizer

IV.A. Simulation Test Protocol

SIMULATION.

Setup: Create a synthetic frequency landscape for 20–68 qubits on a 2D grid. Define the four error components (dephasing, relaxation, stray coupling, pulse distortion) using physically motivated functions. Plant known T_1 hotspots, collision zones, and pulse-distortion penalties.

Test 1 — Error Model Accuracy: Generate random frequency configurations; compute predicted errors via the 4-component model; compare against "ground truth" errors from the full Hamiltonian simulation. Measure R^2 and residual bias.

Test 2 — Weight Training: Generate $\sim$ 1000–6500 synthetic CZXEB measurements at varied frequency configurations. Train the 16 weights via least-squares. Measure: prediction accuracy on held-out configurations, weight stability across training runs.

Test 3 — Inner-Loop Optimizer Comparison: Run the Snake with each candidate inner loop (grid search, Nelder-Mead, CMA-ES) at scope $S = 1, 2, 4, S_{\max}$. Measure: final cycle error, runtime, reproducibility across 10 random seeds.

Test 4 — Scaling: Test on grid sizes $N = 9, 17, 37, 68, 105$. Measure cycle error and runtime scaling. Compare against published Google data (Table 4).

Test 5 — Healing: After Snake optimization, introduce 5–10 outlier qubits (simulating TLS events). Run the healing procedure. Measure: outlier suppression fraction, runtime vs. full re-optimization.

EXPERIMENT.

Protocol (5–10 qubits): Measure the four characterization datasets (flux sensitivity, T_1 spectra, ZZ couplings, pulse distortion). Run the Snake optimizer at $S = 1$ and $S = 2$. Measure CZXEB cycle errors before and after optimization.

Protocol (20+ qubits): Full Snake deployment with weight training from CZXEB data. Compare against manual frequency assignment by an expert.

IV.B. Benchmark Metrics for Gap 3

Metric	**Definition**	**Target**
Cycle error reduction	Optimized / unoptimized median cycle error	$> 2\times$
Error model R^2	Predicted vs. measured cycle error correlation	> 0.8
Outlier fraction	Gates with error $> 2\times$ median after optimization	$< 5\%$
Runtime ($S = 2$, $N = 68$)	Wall-clock for full optimization	< 5 min
Weight stability	Std. dev. of weights across 10 training runs	$< 20\%$
Scaling exponent	Runtime $\propto N^{\beta}$; measure β	$\beta < 2$

Table 4: Benchmark metrics for Snake optimizer testing.

V. Testing Gap 5: TLS Forecasting

V.A. Simulation Test Protocol

SIMULATION.

This is the most critical simulation test because TLS behavior on real devices is slow to observe (hours–days). The simulation enables years of synthetic TLS data in minutes.

Setup: Create a `TLSLandscape` with 5–20 synthetic TLS per qubit, each with:

- Initial frequency drawn from $\text{Uniform}(4, 7)$ GHz
- Coupling $g/2\pi \sim \text{LogNormal}(0.7, 0.5)$ MHz
- Diffusion constant $D = 2.2$ MHz/h$\frac{1}{2}$ (with per-TLS scatter $\pm 30\%$)
- Switching rate $r \sim \text{LogUniform}(10^{-4}, 10)$ Hz
- Linewidth $\gamma \sim \text{LogNormal}(-1, 0.5)$ MHz

Run the landscape forward in simulated time, generating synthetic T_1 measurements.

Test 1 — Algorithm Accuracy: For each of the 6 proposed algorithms, measure:

- **Mean Absolute Error (MAE)** of predicted TLS frequency vs. actual, at horizons $\tau = 1, 4, 12, 24$ h.
- **Coverage probability**: fraction of true TLS positions within predicted 95% confidence interval.
- **False alarm rate**: predicted TLS-in-band events that don't materialize.
- **Miss rate**: actual TLS-in-band events not predicted.

Test 2 — Frequency Allocation Quality: Integrate each forecaster with a frequency allocator. Measure the fraction of time qubits spend with T_1 below a threshold (< 20 μs) over 100 simulated hours.

Test 3 — Robustness to Model Mismatch: Vary the true D by $\pm 50\%$ from the assumed value. Measure degradation of forecast accuracy.

Test 4 — Cosmic Ray Injection: Every ~ 600 s, randomly scramble 3–5 TLS frequencies simultaneously. Measure detection latency and recovery time for each algorithm.

Test 5 — Scaling: Test with $N_{\text{TLS}} = 5, 20, 100, 500$ per qubit. Measure computational cost scaling.

EXPERIMENT.

Protocol (1–3 qubits, 1–2 weeks):
1. Perform swap spectroscopy at the start of each cooldown to catalog TLS.
2. Monitor T_1 continuously at 1 measurement / 10 seconds at the operating frequency.
3. Every 4 hours, perform a T_1-vs.-frequency scan (50 points, 5 min).
4. Run each forecasting algorithm in parallel on the accumulated data.
5. Measure: which algorithm best predicts the next 4-hour T_1 trajectory?

Protocol (5–10 qubits, 1 month):
1. Deploy the best 2–3 algorithms from the single-qubit test.
2. Integrate with the Snake optimizer's relaxation-rate component.
3. Measure: improvement in median gate fidelity when TLS forecasting is active vs. static frequency assignment.

V.B. Benchmark Metrics for Gap 5

Metric	**Definition**	**Target**
MAE (4 h horizon)	Mean absolute TLS frequency prediction error	< 5 MHz
Coverage (95% CI)	Fraction of true positions within predicted interval	$> 90\%$
Miss rate	TLS-in-band events not predicted / total events	$< 10\%$
False alarm rate	False TLS-in-band predictions / total predictions	$< 20\%$
T_1 time-below-threshold	Fraction of time with $T_1 < 20$ μs (with forecasting)	$< 2\%$
Detection latency	Time from TLS appearance to detection	< 30 min
Cosmic ray recovery	Time from scramble event to re-optimized frequencies	< 5 min

Table 5: Benchmark metrics for TLS forecasting testing.

VI. Testing Gap 10: Electronics Noise Budget

VI.A. Simulation Test Protocol

SIMULATION.

Setup: Add controllable noise injection to the gate simulation layer:

- DAC amplitude noise: white noise with configurable SNR (50–70 dB).
- Phase noise: colored noise with configurable spectral density (−80 to −140 dBc/Hz at 10 kHz offset).
- Timing jitter: Gaussian jitter with configurable σ (0.1–10 ps).
- Thermal photon noise: configurable mean photon number $|(n) \in [10^{-4}, 10^{-1}]$.

Test 1 — Noise-to-Fidelity Map: Sweep each noise parameter independently while holding others at nominal values. Plot gate fidelity vs.
noise parameter. Identify the noise level at which electronics noise begins to dominate over decoherence.

Test 2 — Combined Budget: Set all noise parameters to values corresponding to each commercial platform (ZI, QM, Qblox, Keysight, QICK). Predict achievable fidelity for each.

Test 3 — Calibration Sensitivity: Determine how much worse the calibration algorithms (DAG tolerances, Snake optimization) perform when electronics noise increases by 2 × and 5 ×.

EXPERIMENT.

Protocol: On the testbed chip with a specific electronics platform:

1. Measure 1Q and 2Q gate fidelity via RB.
2. Intentionally degrade signal quality (add attenuators, reduce DAC resolution via bit masking, add noise sources).
3. Measure fidelity at each degradation level.
4. Compare measured noise-to-fidelity curve against simulation predictions.
5. Validate that decoherence dominates at the nominal operating point.

VII. Integration Test: Full Calibration Stack

VII.A. End-to-End Test

The ultimate test combines all four gap-filling methods into a single calibration run:

SIMULATION (20-qubit processor, 48 simulated hours).

1. $t = 0$: Cold start. Run full DAG bring-up (Gap 1).
2. $t = 0.5$ h: Run Snake optimizer for frequency assignment (Gap 3).
3. $t = 1$ h: Begin continuous operation with TLS forecasting active (Gap 5).
4. $t = 1$–48 h: Run simulated QEC cycles. Every 15 min:
   - DAG `maintain()` checks all nodes.
   - TLS forecaster updates predictions.
   - If TLS predicted in-band: trigger Snake healing.
5. Inject 3 cosmic ray events and 2 sudden T_1 drops.

Success criterion: Median gate fidelity $> 99.5\%$ sustained for 48 h with $< 15\%$ calibration overhead.

EXPERIMENT (5–10 qubit processor, 24+ hours).

Same protocol on real hardware. Compare against baseline (manual calibration or Qibocal `qq auto`).

VIII. Iteration Workflow

VIII.A. Development Cycle

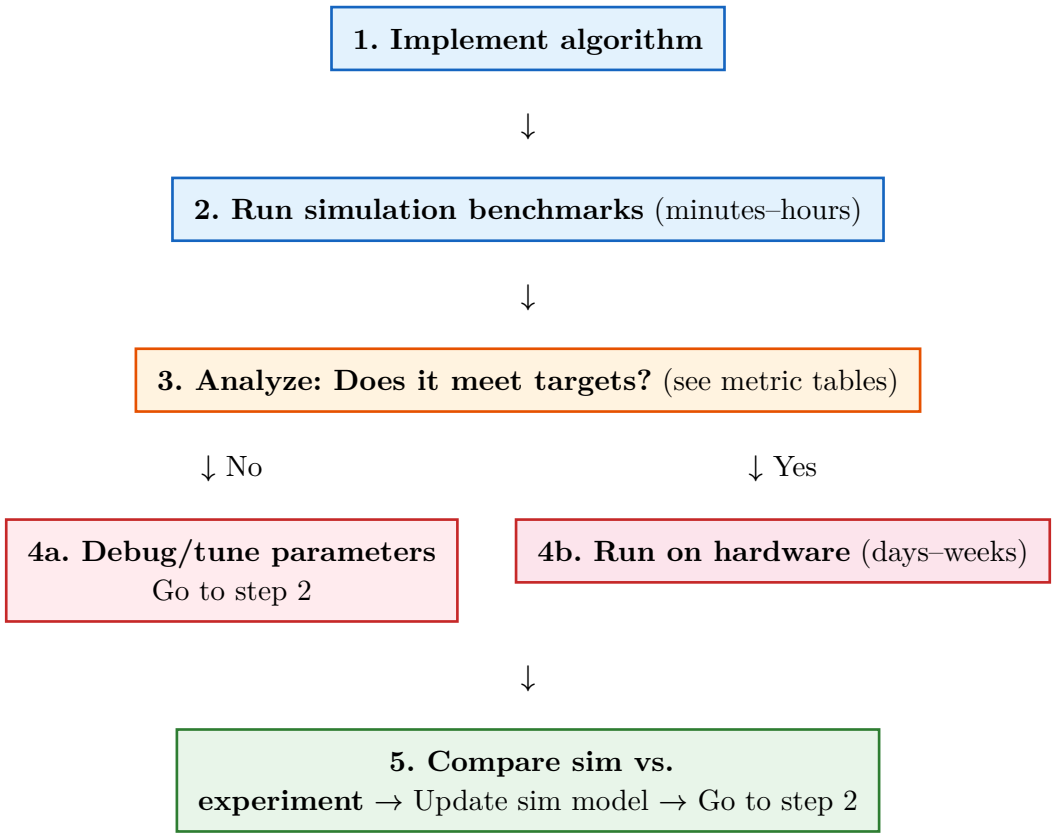

Figure 1: Development iteration cycle. The simulation-first approach minimizes expensive cryostat time.

VIII.B. Key Design Principles

1. **Same API**: calibration code imports `ProcessorBackend` — either `SimulatedProcessor` or `RealProcessor`. No algorithm changes between sim and experiment.
2. **Reproducibility**: all simulation runs use seeded RNG. All experimental data is timestamped and version-controlled.
3. **Metrics-driven**: every test reports the standardized metrics from the tables above. A CI/CD pipeline runs simulation benchmarks on every commit.
4. **Incremental complexity**: start with 1 qubit (test basic calibration nodes), then 2 qubits (test 2Q gates, couplers), then 5–10 (test Snake, TLS), then 20+ (test scaling).

IX. Minimal Hardware Requirements

IX.A. Simulation Track

- Python 3.10+, QuTiP $\geq$ 5.0, scqubits, NumPy, SciPy, matplotlib.
- Desktop CPU: 4+ cores, 16 GB RAM (sufficient for 10-qubit Hamiltonian simulation).
- Optional GPU: NVIDIA with CuPy for parallelized noise simulation at 20+ qubits.

IX.B. Experimental Track

Component	**Minimum Specification**
Qubits	5–10 frequency-tunable transmons with tunable couplers
Control electronics	QICK (ZCU216) or OPX1000 or SHFQC+
Dilution refrigerator	Base < 15 mK, > 400 μW at 4 K
Readout amplifier	TWPA or JPA at MXC, HEMT at 4 K
DC bias sources	24-ch, < 20 nV/$\sqrt{\text{Hz}}$ noise (e.g., QDAC-II)
Compute	Linux server for real-time control and data analysis

Table 6: Minimum experimental hardware for the testbed.

X. Timeline

Phase	Activity	Sim	Exp
1	Build `ProcessorModel` with TLS, drift, noise	Weeks 1–4	—
2	Implement 52-node DAG; test on 1–5 sim qubits	Weeks 3–6	—
3	Implement Snake optimizer; test on 10–20 sim qubits	Weeks 5–8	—
4	Implement 6 TLS forecasting algorithms; benchmark	Weeks 6–10	—
5	Electronics noise model; validate against specs	Weeks 8–10	—
6	Integration test: full stack on 20-qubit sim	Weeks 10–12	—
7	Deploy DAG on 5–10 qubit chip; cold-start bring-up	—	Weeks 10–14
8	Run TLS monitoring campaign (2 weeks continuous)	—	Weeks 14–16
9	Deploy Snake on testbed; measure CZXEB improvement	—	Weeks 16–20
10	Full integration test on hardware (24+ hours)	—	Weeks 20–24
11	Iterate: update sim model from experimental data	Ongoing	Ongoing

Table 7: Development timeline. Simulation phases (blue) and experimental phases (red) overlap after week 10.

XI. Appendix: Software Architecture

```
q-cali/
├── src/
│   ├── simulator/          # Track A: simulation
│   │   ├── transmon.py     # Single transmon model
│   │   ├── processor.py    # Multi-qubit processor model
│   │   ├── tls.py          # TLS landscape simulation
│   │   ├── noise.py        # Noise models (flux, readout, electronics)
│   │   └── backend.py      # SimulatedProcessor backend
│   ├── calibration/        # Gap 1: DAG framework
│   │   ├── dag.py          # DAG engine (maintain, diagnose)
│   │   ├── nodes/          # Individual calibration nodes
│   │   │   ├── spectroscopy.py
│   │   │   ├── rabi.py
│   │   │   ├── ramsey.py
```

```
│   │   │   ├── drag.py
│   │   │   ├── readout.py
│   │   │   ├── cz.py
│   │   │   └── ...
│   │   └── tolerances.py    # Default tolerance/timeout values
│   ├── optimizer/           # Gap 3: Snake optimizer
│   │   ├── snake.py         # Snake outer loop
│   │   ├── error_model.py   # 4-component error estimator
│   │   ├── inner_loop.py    # NM / CMA-ES / grid search
│   │   └── collision.py     # Frequency collision rules
│   ├── tls_forecast/        # Gap 5: TLS forecasting
│   │   ├── particle_filter.py
│   │   ├── hmm.py
│   │   ├── changepoint.py   # Algorithm 5 (most likely Google)
│   │   ├── gp_forecast.py
│   │   ├── neural_forecast.py
│   │   └── fpga_tracker.py
│   └── backend/             # Hardware abstraction
│       ├── base.py          # ProcessorBackend ABC
│       └── hardware.py      # RealProcessor backend (wraps Qibocal/QICK)
├── tests/                   # Unit + integration tests
├── benchmarks/              # Benchmark scripts (simulation)
├── notebooks/               # Jupyter analysis notebooks
├── docs/                    # PDF survey + gap solutions
├── pyproject.toml
└── README.md
```